
B28: SQL SuperHero Program : Session 35

1 message

Vishal Jaiswal <vishaldbdev@gmail.com>
Bcc: satishhattekar1997@gmail.com

22 April 2025 at 22:39

Triggers:

- Triggers are database event.
- they are fired on insert/update and delete.
- We **do not have to call triggers explicitly, it done automatically when insert/update/delete happen** and we wrote a trigger on that.

Question: I wrote a select statement and it fetch 500 records. which trigger will fire?

Answer: NO. Trigger will not fire, because it only fire on DML,i.e insert/update/delete

Why we need Triggers:

Suppose we have a table and we want to audit or we want to log data if someone wants to **insert/update/delete** any record. I want to keep log the data, who did, when did it., what is the old value/what is the new value we use trigger, it fires automatically when any such event occurs and log the data.

Types of Trigger :

It will be Before/After --> 2

It fires on insert/update/delete --> 3

It may be row level/Statement level --> 2

3*2*2 = 12

+ 1 instead of trigger (use for view update)

before insert rowlevel

before update row level

before delete rowlevel

after insert rowlevel

after update rowlevel

after delete rowlevel

before insert Statement level

before update Statement level

before delete Statement level

after insert Statement level

after update Statement level

after delete Statement level

Is trigger advisable :

DEpends on need. As if we have trigger written on table, our DML took much time and performance impacts. so normally it is not prefer until unless it actually needed.

because of taking more time, performance degrades, it is not advisable to use triggers in every scenario.

I asked you to insert 1 row in customer table. you wrote a simple insert statement to insert data in table, but if we have trigger written on that insert , then there is 2 insert. First insert in log table and then insert in main table. si time is doubles, that's why trigger not advisable in every scenario.

/* The below trigger is created on table CUST and it will log any insert/update/delete operation on CUST table in auditing table
This will fire automatically and insert data in auditing table.
*/

```
CREATE TABLE CUST
(
  CUSTID  NUMBER PRIMARY KEY,
  CUSTNAME VARCHAR2(20),
  BALANCE NUMBER(8)
);
```

```
CREATE TABLE AUDITING
(
  TABLENAME  VARCHAR2 (15),
  OP_TYPE     VARCHAR2 (10),
  OLD_BALANCE NUMBER,
  new_balance NUMBER,
  ACCESSED_BY VARCHAR2(200),
  ACCESSED_TIME DATE
);
```

```
CREATE OR REPLACE TRIGGER LOGOPERATION BEFORE
INSERT OR
UPDATE OR
DELETE ON CUST FOR EACH ROW DECLARE vOP_TYPE AUDITING.OP_TYPE%TYPE;
BEGIN
  IF INSERTING THEN
    vOP_TYPE := 'INSERT';
  ELSIF UPDATING THEN
    vOP_TYPE := 'UPDATE';
  ELSE -- MUST BE DELETING
    vOP_TYPE := 'DELETE';
  END IF;
  INSERT
  INTO AUDITING
  (
    TABLENAME,
    OP_TYPE,
    OLD_BALANCE,
    NEW_BALANCE,
    ACCESSED_BY,
    ACCESSED_TIME
  )
  VALUES
  (
    'CUST',
    vOP_TYPE,
    :OLD.BALANCE,
    :NEW.BALANCE,
    USER,
    SYSDATE
  );
END;
```

Syntax

```

create or replace trigger logoperation
before insert or update or delete
on cust for each row
Declare
v_optype auditing.op_type%Type;
Begin
    if inserting then
        v_optype := 'INSERT';
    elsif updating then
        v_optype := 'UPDATE';
    else
        v_optype := 'DELETE';
    End If;

    Insert Into auditing values ('cust', v_optype, :old.balance, :new.balance, user, sysdate);
End;

```

Before or After any one

on this table

Name of trigger

what activity want to log

Row level trigger

Trigger Attribute
 Old = previous
 New = Next

when did

what did

who did

```

select * from cust
no data found ✓

select * from auditing
no data found ✓

```

```
insert into cust values (1,'vishal',5000)
```

1 row(s) inserted.

→ inserted 1 row in cust table

```
select * from cust
```

CUSTID	CUSTNAME	BALANCE
1	vishal	5000

→ We can see data inserted

```
select * from auditing
```

TABLENAME	OP_TYPE	OLD_BALANCE	NEW_BALANCE	ACCESSED_BY	ACCESSED_TIME
CUST	INSERT	-	5000	APEX_PUBLIC_USER	19-APR-25

what did ↓ pv nv → who did → when did

→ Data automatically inserted into auditing table.

Suppose, we update the row in our cust table.

```
select * from cust
```

CUSTID	CUSTNAME	BALANCE
1	vishal	200

→ we update the value from 5000 to 200.

```
select * from auditing
```

TABLENAME	OP_TYPE	OLD_BALANCE	NEW_BALANCE	ACCESSED_BY	ACCESSED_TIME
CUST	INSERT	-	5000	APEX_PUBLIC_USER	19-APR-25
CUST	UPDATE	5000	200	APEX_PUBLIC_USER	19-APR-25

2 rows selected.

what did ↓ pv nv

who

when

→ 1 new row added automatically

insert into cust values (2, 'neetoo', 55000)

Inserted 1 more row in cust table.

```
INSERT INTO CUST VALUES (2, 'NEETOO', 5500)
```

→ 1 more row inserted

1 row(s) inserted.

```
SELECT * FROM auditing
```

TABNAME	OP_TYPE	OLD_BALANCE	NEW_BALANCE	ACCESSED_BY	ACCESSED_TIME
cust	INSERT	-	5500	APEX_PUBLIC_USER	08-SEP-24
cust	INSERT	-	5000	APEX_PUBLIC_USER	08-SEP-24
cust	UPDATE	5000	5500	APEX_PUBLIC_USER	08-SEP-24

3 rows selected.

→ 1 new row
insert
Automatically
becz of
trigger

```
UPDATE CUST SET BALANCE = 50000
```

2 row(s) updated.

→ 2 rows update

```
SELECT * FROM auditing
```

TABNAME	OP_TYPE	OLD_BALANCE	NEW_BALANCE	ACCESSED_BY	ACCESSED_TIME
cust	INSERT	-	5500	APEX_PUBLIC_USER	08-SEP-24
cust	UPDATE	5500	50000	APEX_PUBLIC_USER	08-SEP-24
cust	UPDATE	5500	50000	APEX_PUBLIC_USER	08-SEP-24
cust	INSERT	-	5000	APEX_PUBLIC_USER	08-SEP-24
cust	UPDATE	5000	5500	APEX_PUBLIC_USER	08-SEP-24

5 rows selected.

2 new
entry logged
becz of
trigger

Everything
logged
Automatically

I fire a statement and that statement update 500 rows. How many times trigger will fire?

--> If we have a trigger written on update then this trigger fire. **If statement level then fire 1 time and if row level then fire 500 times.**

Example:

```
UPDATE CUST SET BALANCE = 55000
```

3 row(s) updated.

3 new insert
↓
bec 3
rows
updated
in
cust table

SELECT * FROM auditing

TABLERNAME	OP_TYPE	OLD_BALANCE	NEW_BALANCE	ACCESSED_BY	ACCESSED_TIME
cust	INSERT	-	5500	APEX_PUBLIC_USER	08-SEP-24
cust	UPDATE	50000	55000	APEX_PUBLIC_USER	08-SEP-24
cust	UPDATE	50000	55000	APEX_PUBLIC_USER	08-SEP-24
cust	UPDATE	50000	55000	APEX_PUBLIC_USER	08-SEP-24
cust	UPDATE	5500	50000	APEX_PUBLIC_USER	08-SEP-24
cust	UPDATE	5500	50000	APEX_PUBLIC_USER	08-SEP-24
cust	INSERT	-	50000	APEX_PUBLIC_USER	08-SEP-24
cust	INSERT	-	5000	APEX_PUBLIC_USER	08-SEP-24
cust	UPDATE	5000	5500	APEX_PUBLIC_USER	08-SEP-24

9 rows selected.

=====

If we run a DML and fire Rollback, is data rollback from log table as well? What happened with data inserted via trigger?

Yes, Rolledback, because of ACID property.

1. Out table Auditing has 9 records. (Check above image)
2. I wrote one update statement, which will update 1 data in cust table, so trigger fire 1 time
3. If we check then our auditing table will be 10 records now
4. We can see the below example

```
declare  
    l_count number;
```

```
Begin
```

```
    update cust set balance = 75000 where custid = 1;
```

```
    select count(*) into l_count from auditing;
```

```
    dbms_output.put_line(l_count);
```

```
    rollback;
```

```
End;
```

```
Statement processed.
```

```
10
```

→ we update 1 row

10 records

Now, we did rollback. So what will happen?

Yes data will be rollback from audit table as well. This is ACID. Everything completed or nothing. Insert in Auditing table because of trigger, which was fired automatically when we update cust table. Now when we rollback, everything rollback.

TABLERNAME	OP_TYPE	OLD_BALANCE	NEW_BALANCE	ACCESSED_BY	ACCESSED_TIME
cust	INSERT	-	5500	APEX_PUBLIC_USER	08-SEP-24
cust	UPDATE	50000	55000	APEX_PUBLIC_USER	08-SEP-24
cust	UPDATE	50000	55000	APEX_PUBLIC_USER	08-SEP-24
cust	UPDATE	50000	55000	APEX_PUBLIC_USER	08-SEP-24
cust	UPDATE	5500	50000	APEX_PUBLIC_USER	08-SEP-24
cust	UPDATE	5500	50000	APEX_PUBLIC_USER	08-SEP-24
cust	INSERT	-	50000	APEX_PUBLIC_USER	08-SEP-24
cust	INSERT	-	5000	APEX_PUBLIC_USER	08-SEP-24
cust	UPDATE	5000	5500	APEX_PUBLIC_USER	08-SEP-24

9 rows selected.

→ 10th record is rollback

STATEMENT LEVEL TRIGGER

The default triggers are statement level triggers and we are not using for each row statement in the statement level trigger.

Syntax :

Create or replace Trigger Trigger name

BEFORE/AFTER INSERT/UPDATE/DELETE ON table_name

Begin

SQL statements;

END;

- There is No :OLD and :NEW things with statement level triggers
- Statement level triggers are default triggers which can be created once you create a trigger without using for each row statement.
- It is firing once for every triggering event.
- It is also firing once even if no rows are affected.

Step 1 : Create table named Employee

```
create table Employee
(emp_id number,
ename varchar2(100)
);
```

Step 2 : Insert values in employee table

```
insert into Employee values (1,'vishal');
insert into Employee values (2,'jaiswal');
```

Step 3 : Write the Code for Trigger

```
create or replace trigger check_Statement_Level
before update
on Employee
-- Before updating the table named Employee trigger needs to be fired
```

```
begin
DBMS_OUTPUT.PUT_LINE('Trigger Fired Before Update');
end;
```

```
create or replace trigger check_statement_lvl_trigger
before update on employee
Begin
    dbms_output.put_line('Trigger fired before update');
End;

Trigger created.
```

} Statement level
(Removed FOR EACH ROW)

```
create or replace trigger check_statement_lvl_trigger
before update on employee for each row
Begin
    dbms_output.put_line('Trigger fired before update');
End;

Trigger created.
```

} Row level

```
update employee set empname = 'neetu'
```

```
2 row(s) updated.
Trigger fired before update
```

only 1 time

→ Statement level

```
update employee set empname = 'neetu'
```

```
2 row(s) updated.
Trigger fired before update
Trigger fired before update
```

2 times

→ Row level

```
update employee set empname = 'maurya' where empid = 3
```

```
0 row(s) updated.  
Trigger fired before update
```

→ statement level .
even, no row impact. its fired

```
update employee set empname = 'maurya' where empid = 3
```

```
0 row(s) updated.
```

→ Row level

→ No row impact, Not fired

----> Even if no rows impact and we have a statement level trigger, the trigger will fire. If we have Row level trigger and no rows impact then the trigger will not fire.

=====

Can we make sure of the sequence of triggers? Like if 2 triggers are written on the table, which one will fire first?

NO. Normally we can not. Because triggers are database events that are called automatically, we are not calling them explicitly. But if 1 trigger is after trigger and other is before trigger then obviously before trigger will run first and after will later. But if both triggers are the same "either before" or "after", you can not force them to run in sequence normally.

Oracle allows more than one trigger to be created for the same timing point, but it has never guaranteed the execution order of those triggers. The Oracle 11g trigger syntax now includes the FOLLOWS clause to guarantee execution order for triggers defined with the same timing point

```
CREATE TABLE trigger_test (  
  id      NUMBER,  
  description VARCHAR2(50)  
);
```

```
CREATE OR REPLACE TRIGGER trigger_test_trg_1  
BEFORE INSERT ON trigger_test  
FOR EACH ROW  
BEGIN  
  DBMS_OUTPUT.put_line('TRIGGER_TEST_TRG_1 - Executed');  
END;
```

```
CREATE OR REPLACE TRIGGER trigger_test_trg_2  
BEFORE INSERT ON trigger_test  
FOR EACH ROW  
BEGIN  
  DBMS_OUTPUT.put_line('TRIGGER_TEST_TRG_2 - Executed');  
END;
```

--> 2 same before trigger is created.

```
1 INSERT INTO trigger_test VALUES (1, 'ONE');|
```



```
1 row(s) inserted.  
TRIGGER_TEST_TRG_2 - Executed  
TRIGGER_TEST_TRG_1 - Executed
```

✓ 2 is executed first

If you see the above screen shot, trigger 2 is executed first, if I want Trigger 1 to execute before trigger 2, we have to use **FOLLOWS**. let me change the code of trigger 2.

```
CREATE OR REPLACE TRIGGER trigger_test_trg_2  
BEFORE INSERT ON trigger_test  
FOR EACH ROW  
FOLLOWS trigger_test_trg_1  
BEGIN  
    DBMS_OUTPUT.put_line('TRIGGER_TEST_TRG_2 - Executed');  
END;
```

```
1 CREATE OR REPLACE TRIGGER trigger_test_trg_2  
2 BEFORE INSERT ON trigger_test  
3 FOR EACH ROW  
4 FOLLOWS trigger_test_trg_1  
5 BEGIN  
6     DBMS_OUTPUT.put_line('TRIGGER_TEST_TRG_2 - Executed');  
7 END;|
```



```
Trigger created.
```

FOLLOWS
Keyword force

FOLLOWS keyword forced to follow the sequence. Now if trigger 2 is invoked, it will run trigger 1 first and then trigger 2.

```
insert into trigger_test values (1, 'one')
```



```
1 row(s) inserted.  
trigger_test_trg_1 - Executed  
trigger_test_trg_2 - Executed
```

✓ 1 executed first

